AnnoForge: Bootstrapping Complex Annotation from Guidelines by Recursive Self-Improvement of the Annotation Harness

Anonymous ACL submission

Abstract

Annotating text with a complex ontology takes both an understanding of the structures involved and close adherence to a long guideline, and at the start of a project there is too little annotated data to train a parser. We present AnnoForge, which bootstraps an annotator from the guideline and a few annotated sentences without fine-tuning. A builder LLM writes a seed harness of model steps from the guideline alone, then improves it by recursive self-improvement on groups of five gold sentences: it summarizes the errors a fine-grained judge reports, reflects on their causes, and edits the instructions, adds rule steps and writes tools over resource files such as PropBank, subject only to machine checks. On the Chinese and English test documents of the UMR parsing shared task, ten rounds (50 annotated sentences) raise a harness run by Claude Haiku from 46.4 to 52.4 (Chinese) and from 40.9 to 52.2 (English) AnCast++ F, and to 42.9 and 38.0 on the official juːmætʃ score, above the best fine-tuned shared-task system on the same documents (38.8, 27.3). The builders write PropBank lookup tools when told only that frame files exist, and the learned harness transfers unchanged to other annotators (Sonnet, GPT-6.1-sol), while the choice of generator and critic models matters far less than the harness. On TimeML the harness learns which events and times to link but not how they are related, and it adopts the link density of whatever gold it is trained on. A guideline plus a handful of examples suffices to learn the conventions of a complex scheme and the tools it needs; judgments that no resource supports remain the model's.

1 Introduction

Annotating text with a rich ontology takes two things. One is an understanding of the structure being annotated: what an event is, how its participants and time are related, what counts as the same entity. The other is the given codebook, the guideline that fixes the labels, the conventions and the hard cases, and that annotators apply rather than invent (Halterman and Keith, 2026b; Pustejovsky and Stubbs, 2012). Complex ontologies such as Uniform Meaning Representation (UMR; Van Gysel et al., 2021; Bonn et al., 2024) and TimeML (Pustejovsky et al., 2003; Saurí et al., 2006) are hardest to annotate at the beginning, when there is little annotated data. With enough data a parser can be trained; the systems of the first UMR parsing shared task are fine-tuned models (Štěpánek et al., 2026; Heinecke and Asadullah, 2026; de Vergnette and Amblard, 2026). With a few annotated sentences that route is closed. Large language models (LLMs) offer a way to bootstrap: pretraining gives them a working understanding of meaning and of structured formats (PENMAN graphs, inline XML), and they can be given the guideline itself.

Earlier work has prepared this route. LLMs label text well when given task descriptions or codebooks (Gilardi et al., 2023; He et al., 2024; Halterman and Keith, 2026a), reasoning models can follow annotation guidelines, and an LLM can keep improving a guideline from its own errors against gold (Kim et al., 2026; Kim and Yoon, 2026; Bibal et al., 2025). These refinement studies, however, use simple and direct tasks such as named-entity recognition, whose guidelines fit in one prompt. Bootstrapping a complex scheme is different. The UMR guideline runs to 366 thousand characters, and annotating also needs outside resources such as PropBank frames (Palmer et al., 2005). A model given only the long guideline cannot follow all of it: information in the middle of a long context is used poorly (Liu et al., 2024), accuracy falls as inputs grow (Levy et al., 2024) and as instructions accumulate (Jaroslawicz et al., 2025), and agents lose rule details of long procedure manuals (Panavas et al., 2026).

We argue that bootstrapping a complex annotation needs more than a better guideline: the guideline has to be broken into a chain of steps, and

the steps need tools, so that the model's own abilities are used where they are strong and supported where they are not. We present AnnoForge (Figure 1), which builds an annotator from the guideline and a few annotated sentences by recursive self-improvement (RSI) of its *harness*: the instructions of each step (skills), the steps themselves and the tools they call. A builder model writes a seed harness from the guideline alone; then, five sentences at a time, it annotates gold sentences with the current harness, summarizes the errors, reflects on their causes and proposes edits, new steps and new tools, which are admitted by machine checks. The result annotates from instructions and a handful of examples, with no fine-tuning.

We evaluate AnnoForge on Chinese and English UMR, using the test documents of the UMR parsing shared task (Štěpánek et al., 2026), and on TimeML with the TempEval-3 platinum test set (UzZaman et al., 2013), with Claude Haiku 5.5 and GPT-6.1-sol as annotators and Claude Sonnet 5.5 as builder. Our contributions:

- A guideline-and-harness optimization method for bootstrapping complex annotation, in which skills, steps and tools are all editable and a fine-grained judge tells the builder which part of the scheme each error belongs to (§3); we also release timemljudge, a per-check judge for TimeML.
- Evidence that it works without fine-tuning: from ten groups of five annotated sentences, the learned harness improves UMR annotation on held-out shared-task documents by +6.1 (Chinese) and +11.3 (English) AnCast++ F, and exceeds the fine-tuned shared-task systems on the same documents on the official score (§5.1).
- Evidence of continued evolution: what is learned on one group carries over to unseen documents and to other annotating models; the builders invent and later revise PropBank tools when told only that frame files exist, and turn recurring convention errors into code (§5.2, §5.3). The analysis also shows the limit: the loop learns the conventions of the gold it sees, which pairs of events to link but not their temporal relation, and from sparse gold, to link less.

2 Related Work

LLMs as annotators, guidelines as input. LLMs can replace or assist crowd workers on many labeling tasks (Wang et al., 2021; Gilardi et al., 2023; He et al., 2024; Ziems et al., 2024; Tan et al., 2024), but they follow a scheme's conventions poorly out of the box: as AMR analyzers (Ettinger et al., 2023), as annotators of document-level event relations (Wei et al., 2024), and with real codebooks, whose underspecified concepts they silently accept (Halterman and Keith, 2026a,b); they help experts more than they replace them (Gu et al., 2026). Guidelines help when models are trained to follow them (Sainz et al., 2024; Srivastava et al., 2025) or replace examples in context (Huang et al., 2025). Guidelines can also be learned: Guideline Learning induces rules from errors (Pang et al., 2023), in the tradition of error-driven rule learning (Brill, 1995); LLMs revise annotation guidelines against gold (Bibal et al., 2025; Kim et al., 2025), and Kim et al. (2026) let a moderator model write one principle at a time into an NER guideline and keep it when development F1 improves, while Kim and Yoon (2026) refine NER instructions from the disagreements of several LLM annotators. Codebook work revises coding instructions from expert feedback or model disagreement (He et al., 2026; Yuan et al., 2026). These methods change instruction text for flat labeling tasks. Structured schemes such as UMR and TimeML are annotated by trained parsers when data exist (Chun and Xue, 2024; Markle et al., 2026; Bethard, 2013; Heinecke and Asadullah, 2026), by converting other resources (Gamba et al., 2025), or by people revising model drafts (Sun et al., 2024); we bootstrap them from the guideline and a few examples, and the object we change is the whole harness.

RSI, prompt optimization, self-improving agents and harnesses. Systems that improve the code that improves them go back to the Gödel machine (Schmidhuber, 2003); with LLMs, a model can improve its own scaffolding (Zelikman et al., 2024), an agent can rewrite its own logic (Yin et al., 2025), and archives of self-modified agents can be searched (Zhang et al., 2026b; Novikov et al., 2025). Prompts can be searched and evolved (Zhou et al., 2023; Yang et al., 2024; Fernando et al., 2024; Guo et al., 2024), edited along textual "gradients" or execution traces (Pryzant et al., 2023; Yuksekgonul et al., 2025; Cheng et al., 2024), compiled with demonstrations (Khattab et al., 2024; Opsahl-Ong et al., 2024), evolved by reflection (Agrawal et al., 2026), or grown as playbooks (Zhang et al.,

2026c; Suzgun et al., 2026); with little data they can lose to fixed prompts (Singh, 2026). Agents also learn skills (Wang et al., 2024a; Zheng et al., 2025), make tools (Cai et al., 2024; Qian et al., 2023; Yuan et al., 2024; Wang et al., 2024b), and search over workflows (Hu et al., 2025; Zhang et al., 2025; Zhuge et al., 2024). Recent work optimizes the harness of coding and web agents itself (Lee et al., 2026; Zhang et al., 2026a; Lin et al., 2026a; Xia et al., 2026), and finds that gains compound only with regression control (Wang et al., 2026) and that writing useful updates and benefiting from them dissociate across model tiers (Lin et al., 2026b). Self-feedback rarely helps without an external signal (Madaan et al., 2023; Shinn et al., 2023; Huang et al., 2024; Kamoi et al., 2024). We bring this line of work to annotation, where gold is scarce but structured: an aligned gold annotation and a judge that sorts each error into a part of the scheme give the builder far more than a pass/fail reward, and the improvement loop plays the role of the moderation step of a human annotation project (Pustejovsky and Stubbs, 2012).

3 AnnoForge

3.1 Overview

AnnoForge turns an annotation guideline and a few annotated sentences into an annotator, without training any model. Nothing in it is specific to a scheme: a benchmark supplies the guideline, the output format, a parser for that format, and an evaluator; the rest is shared (Figure 1).

Annotator. An annotator is a harness $H = (S, T, G, R)$ run by language models: *skills* S, the instructions of each model step; *tools* T, typed Python functions a step may call; a *chain* G, the order of the steps and the model that runs each; and *rule steps* R, code that rewrites an annotation without a model. A document is annotated sentence by sentence. Each step sees the document header, the five preceding sentences with their final annotations (a registry of the document so far), the sentence, the current annotation and the open findings. A *rewrite* step writes the complete annotation of the sentence, a *check* step returns findings (a problem and its fix) for the next rewrite step, a rule step is code. The final annotation is the last one that parses. Every step receives the same fixed output format; nothing else is fixed.

Seed. The builder model reads the guideline and writes the seed harness H_0: three steps, `generate` (writes the annotation), `critique` (lists problems) and `finalize` (rewrites the annotation with the findings), each a checklist of about 8,000 characters (the builder is asked for at most 8,000). It sees no annotated data and the seed has no tools.

Recursive self-improvement. Training data come as groups of five consecutive sentences of one document. Round n turns H_{n-1} into H_n with group n (§3.3): annotate the group, score it with a fine-grained judge, let the builder summarize the problems, reflect on their causes and propose operations on skills, steps and tools, admit the operations that pass machine checks, and annotate the group again. Each H_n is also tested on held-out documents, which the builder never sees.

3.2 Two benchmarks

UMR. A UMR annotation of a sentence (Van Gysel et al., 2021; Bonn et al., 2024) is a rooted graph in PENMAN notation whose nodes are concepts: events with PropBank-style senses (Palmer et al., 2005) or language-specific lemmas, entities, named-entity types, and abstract rolesets such as `have-org-role-92`; edges are participant roles (`:ARG0`, ...) and non-participant roles (`:temporal`, `:manner`, ...); attributes give aspect, polarity, mode, and the person and number of referring expressions. Each node is aligned to the words it comes from. A document-level graph per sentence adds temporal relations between events and times, modal relations between conceivers and events (who asserts what with which strength), and coreference with earlier sentences. The guideline (Van Gysel et al., 2021) has 365,742 characters.

TimeML. A TimeML annotation (Pustejovsky et al., 2003), following the TimeML 1.2.1 guidelines (Saurí et al., 2006) as in TimeBank 1.2 (Pustejovsky et al., 2006) and TempEval-3 (UzZaman et al., 2013), marks time expressions (`TIMEX3`, with a type and a normalized value such as `2013-W11`), events (`EVENT`, with a class and the tense, aspect, polarity and modality of its instance), and temporal links (`TLINK`) of thirteen relation types between events, times and the document creation time, across sentences. The guideline has 115,146 characters. An annotation of a sentence is the sentence with inline tags followed by its TLINK lines.

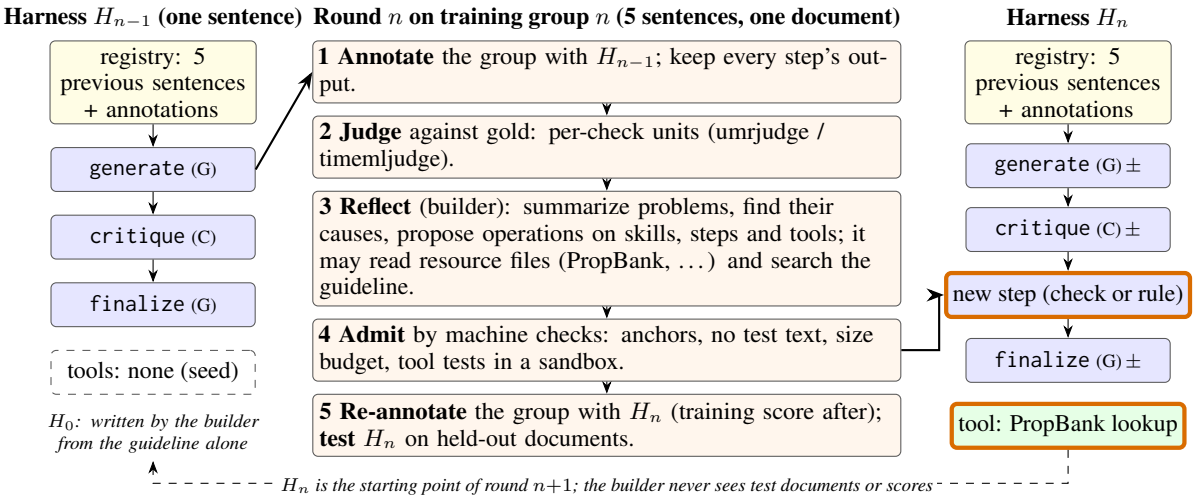


Figure 1: ANNOFORGE. Left: the seed harness annotates one sentence at a time; each step sees the previous five sentences with their annotations. Middle: one round of recursive self-improvement on a group of five gold sentences. Right: the next version; skills are edited (±), and steps and tools may be added (orange), e.g. the PropBank lookup tools the builders wrote after learning that frame files exist (§5.3). G and C are the generator and critic models of a configuration.

Judges. The builder needs to know which part of the scheme an error belongs to. For UMR we use umrjudge: AnCast++ (Sun and Xue, 2024; Sun et al., 2025) aligns predicted and gold nodes, and 25 checks in 11 groups compare the aligned graphs (nodes, senses, attributes, arguments, role labels, names, reentrancy, and the document-level temporal, modal and coreference relations; Appendix B). For TimeML we built timemljudge on the same design from earlier evaluations: TempEval-3's extent and attribute measures for times and events (UzZaman et al., 2013), and the TempEval-2 split of temporal links by what they connect (Verhagen et al., 2010), event–time in one sentence or across sentences, event–document time, event–event in one sentence or across, and time–time, each scored for which pairs are linked and for the relation of the pairs linked on both sides. Both judges return *units*, each with a verdict (correct, wrong, missing, extra), the sentence, and the gold and predicted elements.

3.3 The harness and its improvement

Representation. A harness version is a directory of plain files: `harness.yaml` (the chain: each step's name, kind, model and tools), one Markdown skill per model step, one Python module per tool, one per rule step. A version is never changed after it is written; round n copies H_{n-1} and edits the copy. A configuration names two models: the generator G runs `generate` and `finalize`, the critic C runs `critique`; a step the builder adds names which of the two runs it.

Resources. The annotating steps see only their input. Tools may read resource files through one module; the builder is told what files exist (for UMR: the Chinese and English PropBank frame files, the UMR lists of abstract rolesets, relations and attribute values, and the guideline; for TimeML: the guideline) and may look into them while it reflects. It is not told what to build: in the seed there are no tools, and a PropBank lookup has to be invented.

One round. (1) *Annotate*: H_{n-1} annotates the five sentences of group n; the sentences before the group are given with their gold annotations as context. The trace keeps every step's output and findings. (2) *Judge*: the group is scored and the judge lists the units that are not correct. (3) *Reflect*: one builder call receives the output format, the harness (chain, all skills with their sizes, all tools), the resource manifest, the previous rounds' changes and training scores (never a test score), and for each sentence its gold annotation, every step's output and its incorrect units. It returns *problems* (error patterns with evidence and the step to fix), a *reflection* (why they happen and what to change), *operations*, and *suggestions* it cannot implement. The builder can list and read resource files and search the guideline during the call. (4) *Admit*: operations are applied in order to a copy of H_{n-1} if they pass machine checks. (5) *Re-annotate* the group with H_n, and test H_n on held-out documents.

Operations. `edit_skill` modifies, inserts or deletes text at an anchor that must occur exactly once; `rewrite_skill` replaces a skill, to consolidate it; `add_step` inserts a check, rewrite or rule step after an existing step, `delete_step` removes one (at most five model steps); `add_tool`, `replace_tool` and `delete_tool` manage tools, and `set_step_tools` decides which steps may call which tool.

Admission. Every operation is checked by machine; there is no reviewer model, so that tools and steps are not blocked by a second opinion. No text may copy eight consecutive characters (Chinese) or six consecutive words (English) of a test sentence. A skill may not grow beyond 10,000 characters; an edit that would is rejected, and the builder is told to consolidate with `rewrite_skill`. This keeps the harness compact as rounds accumulate. A tool or rule must import only whitelisted modules (the resource module included), use no file, evaluation or reflection primitives, define one public function with type annotations and a docstring, and pass at least three tests of its own in a separate process; a tool that fails is returned once to the builder with the error. A new step must follow an existing one, and the first step must stay a rewrite step.

4 Experiments

Data. *UMR.* We use the test set of the first UMR parsing shared task (Štěpánek et al., 2026), released with UMR 2.2 (Bonn et al., 2026). Its Chinese and English documents for which we have a cleaned gold annotation [state the source and cleanup of this gold] (13 of the 15 Chinese and 4 of the 5 English test documents) are shuffled with a fixed seed and taken whole while the total stays at most 100 sentences: 7 Chinese documents (99 sentences) and the 4 English documents (95 sentences; two of them are the same text annotated twice, as in the shared task). Training groups are five consecutive sentences of one document, drawn from documents that share no sentence with the test set: Chinese UMR documents 0001–0018, and the fully annotated English documents of the release (most of it is partial AMR conversion without aspect, or Minecraft dialogue). *TimeML.* TempEval-3 (UzZaman et al., 2013): the test documents are 7 of the 20 platinum documents (96 sentences, chosen as above); training groups come from the 13 platinum documents that are not test documents, or, for a second trajectory, from TBAQ-cleaned (TimeBank 1.2 and AQUAINT, the TempEval-3 training data); all follow the TimeML 1.2.1 guidelines (Saurí et al., 2006). Each benchmark has ten training groups, used in a fixed random order, one per round; the builder never sees a test document.

Models. Annotators are Claude Haiku 5.5 (`claude-haiku-5-5`) and GPT-6.1-sol (`openai/gpt-6.1-sol` through OpenRouter); Claude Sonnet 5.5 writes the seed and is the builder in every round. Claude models run with adaptive thinking at the Messages API's default effort (Haiku medium, Sonnet high), GPT with medium reasoning effort. Each configuration was run once (answers are cached, so a repeated call is not a new sample); for UMR we repeated H_0 and H_{10} with fresh samples to estimate run-to-run variation.

Conditions. *Guideline prompt:* one call per sentence whose instructions are the whole guideline, followed by the sentence (no earlier sentences, no examples), with Haiku, Sonnet and GPT. *Seed harness H_0:* the three-step harness written from the guideline, with each of the four generator/critic pairs of Haiku and GPT, and with Sonnet in both roles. AnnoForge*:* ten rounds of RSI with Haiku as generator and critic, testing every version $H_0, \ldots, H_{10}$; the final H_{10} is also run, unchanged, with the other generator/critic pairs and with Sonnet in both roles. For TimeML we run two trajectories, with training groups from the platinum documents that are not test documents (the main one, from the same annotation project as the test set) and from TBAQ-cleaned. All conditions receive the same output-format instructions.

Metrics. UMR: AnCast++ (Sun et al., 2025) sentence-graph F (*F*, the main metric) and its comprehensive score with the document-level relations (*Comp.*), after the cleanup of Chun and Xue (2024), macro-averaged over documents, and the shared task's official juːmætʃ score (Zeman and Gamba, 2026), computed with `compare_umr.pl` against the shared task's own test gold. juːmætʃ maps nodes through the token alignment, so it also scores alignment, and its gold differs from the cleaned gold (Appendix A). TimeML: the official TempEval-3 scorer: temporal awareness TLINK F1 (*TLINK*, the main metric), event and time-expression extent F1 and the value F1 of time expressions. The judges' per-check scores support the analysis.

Prior systems. We re-score earlier systems on exactly our test documents from their released outputs: the two UMR shared-task systems (Heinecke and Asadullah, 2026; de Vergnette and Amblard, 2026), fine-tuned on the shared task's training data, of which Orange is best on both Chinese and English; and the TempEval-3 end-to-end systems (UzZaman et al., 2013), among them ClearTK-2 (Bethard, 2013), the best participant, and TIPSem, the organizers' reference system, which is best on our documents.

Statistics. Differences between two conditions are paired over test documents (UMR: the per-document F; TimeML: the corpus-level TLINK F1 recomputed from per-document counts) with a bootstrap of 20,000 resamples; for UMR we also report a sentence-level bootstrap of per-sentence AnCast scores. With seven or four test documents and one run per configuration, the intervals describe the variation over documents, not over runs.

5 Results

5.1 Main results

Table 1 gives every system on the same test documents.

A long guideline in the prompt is not enough. Given the whole guideline and the sentence, Haiku reaches 30.9 AnCast++ F on Chinese UMR and Sonnet 50.1; on English the prompt baselines are stronger (50.8 and 54.1). Their errors follow the guideline's own notation where the data use another: modal strength written as `:modstr FullAff` inside the graph instead of document-level relations such as `(author :full-affirmative e)`, capitalized attribute values, `:ref-number` for `:refer-number`. Every prompt baseline has a document-level modal score of 0.

The seed harness helps where the guideline is hard to apply. Written from the same guideline, the three-step seed improves over the Haiku prompt baseline on Chinese UMR (46.4 vs. 30.9) and on TimeML (TLINK 39.5 vs. 30.5), where it is above the best TempEval-3 participant on our documents (ClearTK-2, 29.3) and below the organizers' TIPSem (46.3). On English UMR the seed is below the prompt baseline (40.9 vs. 50.8): a one-time compression of the guideline into 18K characters of checklists loses detail.

RSI improves the harness on held-out documents. After ten rounds of five sentences each, H_{10} improves over the seed on both UMR test sets: Chinese F 46.4 → 52.4 (paired document bootstrap +6.1, 95% interval +1.0 to +10.1; sentence level +7.3, 65 of 98 sentences better and 25 worse by more than a point) and English 40.9 → 52.2 (+11.3, +7.7 to +14.9; 72 of 95 better, 18 worse). The comprehensive score, which adds the document level, rises more (34.0 → 47.2; 22.5 → 33.0), and so does the official juːmæʧ score against the shared task's own gold (24.1 → 42.9; 25.9 → 38.0). H_{10} is above the best shared-task system on the same documents on every UMR metric (juːmæʧ 38.8 and 27.3; F 37.7 and 40.3), although that system was fine-tuned on the shared task's training data and we used 50 annotated sentences. Two independent repetitions of H_0 and H_{10} with fresh samples differ by 0.9/2.1 (Chinese) and 1.2/1.5 (English) F, well below the gains. On TimeML, temporal awareness does not improve (39.5 → 39.7); §5.3 shows what does change.

5.2 What RSI learns carries over

Each round improves its own training group, by +14.9 (Chinese), +19.1 (English) and +16.0 (TimeML) points on average (Figure 2, orange). Fitting five sentences would produce such gains on its own. For UMR they are not only that: the test score rises in most rounds (6 of 10 for Chinese, 7 for English), and across rounds the training gain and the test gain are positively correlated, though with ten rounds not significantly (Spearman ρ 0.58 and 0.50, p 0.08 and 0.14). The test curves rise for three to six groups, 15–30 annotated sentences, and then flatten (best 53.5 at round 5 for Chinese, 53.5 at round 7 for English), while the training groups keep improving: later rounds increasingly fix what is particular to a group, a narrative, a list of bond prices, a phrasal verb. Since no edit is tested before it is kept, a few rounds lower the test score, by up to two points.

Across models. H_{10} was built by watching Haiku. Run unchanged with other annotators, it helps them as much: with GPT-6.1-sol as generator and critic, Chinese F rises from 49.1 (seed) to 56.4 (comprehensive and juːmæʧ as with Haiku, Table 1); on the 38 sentences of the two English documents the GPT run of H_{10} completed within our API budget, from 42.7 to 57.1 (Haiku: 40.5 → 52.3 on the same documents); with Sonnet in both roles,

System	UMR Chinese			UMR English			TimeML		
	F	Comp.	ju:mætʃ	F	Comp.	ju:mætʃ	TLINK	Event	Value
Fine-tuned and feature-based systems (re-scored on our test documents)									
UMR shared task: Orange	37.7	34.0	38.8	40.3	24.8	27.3	–	–	–
UMR shared task: Sema	34.7	33.4	29.2	32.1	21.5	21.4	–	–	–
TempEval-3: TIPSem	–	–	–	–	–	–	46.3	82.4	72.5
TempEval-3: ClearTK-2	–	–	–	–	–	–	29.3	76.9	68.0
TempEval-3: NavyTime	–	–	–	–	–	–	27.7	82.1	70.9
Guideline in the prompt, one call per sentence									
Haiku 5.5	30.9	24.1	12.5	50.8	29.9	21.7	30.5	81.5	82.6
Sonnet 5.5	50.1	38.0	24.4	54.1	31.8	28.4	35.1	83.2	80.4
GPT-6.1-sol	n/a	n/a	n/a	n/a	n/a	n/a	39.5	82.8	78.2
Seed harness H_0 (generator / critic)									
Haiku / Haiku	46.4	34.0	24.1	40.9	22.5	25.9	39.5	83.8	78.6
Haiku / GPT	46.3	33.8	25.8	43.0	24.2	26.6	38.1	81.4	70.3
GPT / Haiku	47.8	34.7	23.7	42.3	23.5	26.2	39.6	82.0	75.0
GPT / GPT	49.1	35.5	24.4	42.1	23.8	26.0	39.4	80.8	72.7
Sonnet / Sonnet	48.1	35.1	24.3	43.5	23.6	27.1	38.9	80.0	78.5
Learned harness H_{10} (ANNOFORGE, built with Haiku / Haiku)									
Haiku / Haiku	52.4	47.2	42.9	52.2	33.0	38.0	39.7	84.0	79.3
Haiku / GPT (transfer)	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
GPT / Haiku (transfer)	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
GPT / GPT (transfer)	56.4	49.7	42.4	n/a	n/a	n/a	n/a	n/a	n/a
Sonnet / Sonnet (transfer)	58.9	51.2	42.5	53.1	33.4	38.7	39.5	84.0	78.5
Haiku / Haiku, TimeBank-trained	–	–	–	–	–	–	23.8	84.2	84.1

Table 1: All systems on the test documents (UMR: 99 Chinese and 95 English sentences of the shared task's test set; TimeML: 96 TempEval-3 platinum sentences). F: AnCast++ sentence-graph F; Comp.: AnCast++ comprehensive score with document-level relations; ju:mætʃ: the shared task's official score against its test gold. TLINK: TempEval-3 temporal awareness F1; Event: event extent F1; Value: TIMEX3 value F1. Transfer: H_{10}, built with Haiku annotators, run with other generator/critic pairs. n/a: not run to completion within the API budget for GPT-6.1-sol.

G / C	UMR zh (F)			UMR en (F)			TimeML (TLINK)		
	H_0	H_{10}	Δ	H_0	H_{10}	Δ	H_0	H_{10}	Δ
Haiku / Haiku	46.4	52.4	+6.1	40.9	52.2	+11.3	39.5	39.7	+0.2
Haiku / GPT	46.3	n/a	n/a	43.0	n/a	n/a	38.1	n/a	n/a
GPT / Haiku	47.8	n/a	n/a	42.3	n/a	n/a	39.6	n/a	n/a
GPT / GPT	49.1	56.4	+7.3	42.1	n/a	n/a	39.4	n/a	n/a
Sonnet / Sonnet	48.1	58.9	+10.7	43.5	53.1	+9.6	38.9	39.5	+0.6

Table 2: Generator (G) and critic (C) models (H: Haiku, G: GPT-6.1-sol, S: Sonnet). H_{10} was built with Haiku in both roles; for the other pairs it is used as it is. n/a: not run within the API budget.

Harness (Haiku / Haiku)	UMR Chinese			UMR English		
	F	Comp.	ju:mætʃ	F	Comp.	ju:mætʃ
H_0 (seed)	46.4	34.0	24.1	40.9	22.5	25.9
H_{10}	52.4	47.2	42.9	52.2	33.0	38.0
H_{10} without tools	53.0	48.4	43.5	50.6	32.2	36.6
H_{10} without rule steps	54.3	49.0	43.1	52.6	32.6	37.5
H_{10} with the seed's skills	47.4	43.4	28.7	40.0	21.9	24.9

Table 3: What the parts of H_{10} contribute: the learned harness without its tools, without its rule steps, and with the seed's instructions in place of the learned ones (tools and rules kept).

Chinese 48.1 → 58.9 and English 43.5 → 53.1. What the builder wrote is about the scheme, not about the model it watched.

Generator and critic. At the seed, the choice of models matters much less than the harness (Table 2): the four Haiku/GPT pairs are within three points of each other on every benchmark, close to the run-to-run variation, while RSI adds 6 to 11. A stronger generator helps a little on Chinese (GPT/GPT 49.1 vs. Haiku/Haiku 46.4); swapping only the critic changes little (Haiku/GPT 46.3, GPT/Haiku 47.8), and the mixed pairs show no complementarity beyond that. Consistently, the builders' main complaint about the critic was not its model but its instructions: it "fixed" correct annotations (§5.4).

5.3 How the harness improves

Conventions first, then lookups. Figure 3 breaks the gains down by judge check (Appendix Tables 9–11). For UMR, the largest gains are on conventions that the guideline states differently from the data or not at all: the document-level format and modal relations (Chinese modal

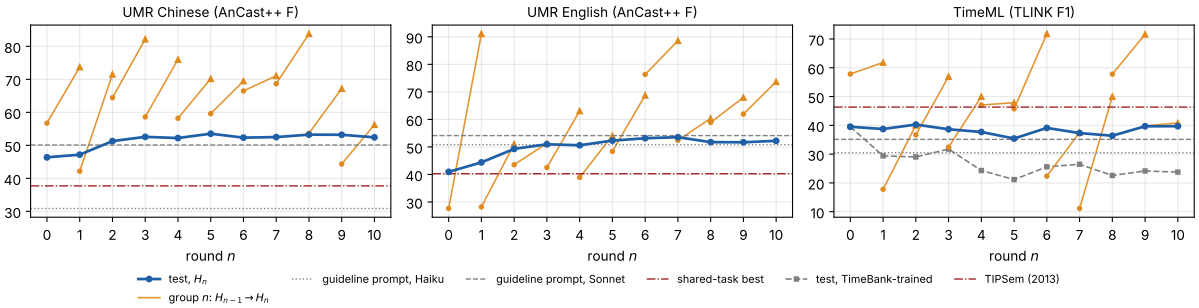


Figure 2: Ten rounds of AnnoForge with Haiku as generator and critic. Blue: test score of H_n. Orange: training group n, annotated by H_{n-1} (circle) and, after the round, by H_n (triangle). Grey squares (TimeML): the trajectory trained on TimeBank/AQUAINT instead of platinum documents. Reference lines: guideline-in-prompt baselines (Haiku, Sonnet) and the best earlier system on the same test documents.

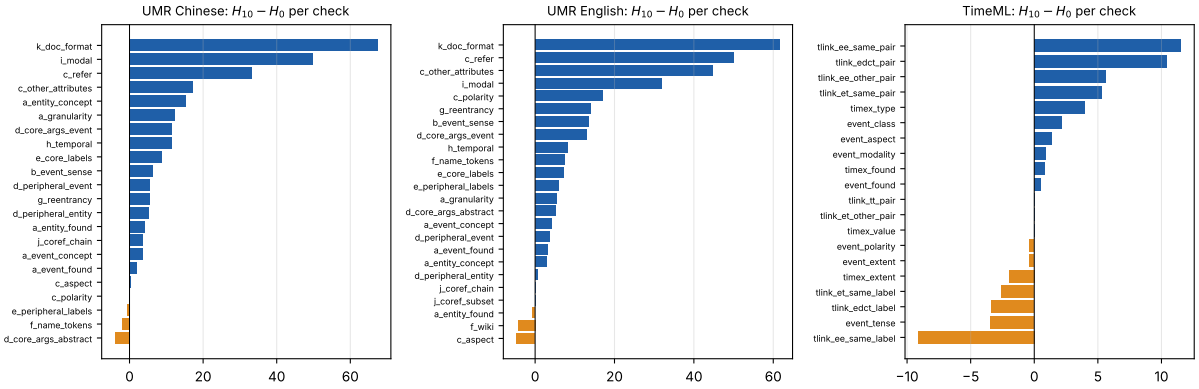


Figure 3: Change of each judge check from H_0 to H_{10} on the test documents (checks with at least six units; Haiku / Haiku; TimeML: platinum-trained trajectory).

0 → 50, English 0 → 32), attribute names and values, entity concepts, granularity. The next largest are PropBank senses (Chinese 88 → 94, English 67 → 80) and core arguments of events (52 → 63, 42 → 55). Aspect and coreference change little, and document-level temporal relations stay low (Chinese 3 → 15, English 5 → 13). The ablation (Table 3) separates the parts: the learned instructions carry most of the gain (with the seed's instructions in place of the learned ones, F falls by −5.0 and −12.3); removing the tools hardly changes F (+0.6, −1.6) but costs exactly the checks the tools serve, senses (Chinese 94 → 78, English 80 → 73) and core arguments (63 → 55, 55 → 47); removing the rule steps changes little (+1.9, +0.4): the learned instructions alone already produce most of what the rules enforce. Differences of one or two points are within the run-to-run variation measured above.

TimeML: pairs, not relations. The fine-grained judge shows what the flat temporal-awareness score hides. The platinum-trained harness links the right pairs much more often: event–document time from 34 to 45, event–event within a sentence from 43 to 55, across sentences from 6 to 12 (pair F); but the relation of a pair linked on both sides gets worse (event–event within a sentence 75 → 66), and temporal awareness needs both. Which pairs to link is a convention of an annotation project; which relation holds is a semantic judgment. Time expressions and events, already near the 2013 systems at H_0, change little.

The builder learns the gold it sees. Trained on TimeBank and AQUAINT instead (grey in Figure 2), whose temporal links are much sparser than the platinum gold and where several five-sentence windows have no link at all, the harness learns to link less: TLINK precision rises (40.0 → 45.7), recall falls (39.0 → 16.0), and temporal awareness drops to 23.8, while time values improve (78.6 → 84.1). The loop does what it is built to do, adapt the annotator to the gold it is given; it cannot tell a project's convention from an accident of a corpus. The builders' suggestions, which the loop

never applies, point the same way: 65 of the 75 suggestions of the three main trajectories are about the gold, 34 of them explicitly about inconsistencies (e.g. `:refer-number` on plural nouns in some English documents but not in others), which no learner can resolve and a human moderator should.

Tools are proposed rarely, but they are proposed. Admission was deliberately lenient: machine checks only, no reviewer. The UMR builders were told only that PropBank files exist, and each wrote a lookup in the first rounds: `lookup_frame` over the Chinese frame files (round 2), which also answers "no frame" with the UMR convention for such predicates, and `roleset_lookup` over the English argument descriptions (round 1), which the builder replaced in round 9 (and refined again in round 10) after noticing that the annotator chose wrong senses from role descriptions alone (the new version reads the XML frames with sense names and falls back from *researchers* to *research*). With H_{10}, 60 % (Chinese) and 39 % (English) of Haiku's calls use the tool. The TimeML builder, whose only resource was the guideline, wrote no tool. Rule steps were added more often (10 for Chinese, 4 for English): the builders turned recurring convention errors into code that normalizes modal labels and constants, moves modal strength from the graph to the document level, and repairs variables and alignments. No model step was even proposed: when one kind of decision needed its own pass, the builders preferred code, or a new section in an existing skill, to an extra call per sentence. Table 4 counts every operation.

Compact by construction. Most operations modify or add instruction text; when a skill approached the 10,000-character budget, the builder consolidated it (7 rewrites for Chinese, 3 for English). The final harnesses have 26.4K (Chinese), 27.9K (English) and 28.8K (TimeML) characters of instructions, against 366K and 115K for the guidelines (Appendix Tables 6–8).

5.4 Case study: one trajectory

The English trajectory shows the order in which the builder worked (Appendix G). Round 1 found that the annotator wrote the guideline's notation (`:ref-number Singular`, modal labels such as `FullAff`, tense metanodes in temporal relations), rewrote the three skills in the data's notation, and added the PropBank tool. Rounds 2–6 worked on the document level: the `(root :modal author)` relation, modal relations for every event with aspect, coreference direction for plural pronouns, and rule steps that clean the document-level graph. Rounds 7–10 turned to semantics: aspect cues such as habitual *would*, phrasal verbs as one roleset (`pick-up-04`), the sense distinctions that led to the new lookup. The critic was a recurring problem: in several rounds the builder found that `critique` had "fixed" correct annotations and `finalize` had applied the fixes, and restricted the critic's checklist, in line with the finding that self-correction without an external signal often hurts (Huang et al., 2024; Kamoi et al., 2024).

6 Conclusion

Complex annotation can be bootstrapped from a guideline and a few examples: without fine-tuning, an LLM that is given the guideline as a chain of steps, tools over the project's resources, and a loop that revises them against a handful of gold sentences, annotates UMR better than the fine-tuned systems of its shared task on the same documents. A long guideline alone is not enough, and neither is a one-time compression of it: the gains come from the harness, its decomposition into steps and its tools, and from recursive self-improvement with a fine-grained judge, which plays the role that moderation plays in a human annotation project. In that loop the builder model learned to edit the guideline, to write the tools the guideline presupposes, and to turn recurring decisions into code.

The same loop shows its limits. What it learns are the conventions of the gold it sees: it learned which TimeML events to link, and, from a sparser corpus, to link less; senses and arguments improved where a tool supplied the frames, but judgments that no resource supports, aspect, coreference and temporal relations, moved little; and its own suggestions are dominated by inconsistencies of the gold. This points to a different division of labor in annotation projects. A guideline can be an executable object that is tested and revised against data from the first annotated sentences on; the model maintains the instructions, tools and checks, and people decide the conventions and adjudicate the inconsistencies it surfaces. Bootstrapping a new scheme or language then starts from a working annotator rather than from an empty corpus.

Benchmark	edit_skill add	edit_skill modify	edit_skill delete	rewrite _skill	add _step	delete _step	add _tool	replace _tool	delete _tool	set step _tools
UMR Chinese	20/22	32/33	14/15	7/7	10/10	0	1/1	0	0	0
UMR English	24/27	32/34	18/18	3/3	4/7	0	1/1	2/2	0	0
TimeML	26/30	68/74	27/29	0	0/1	0	0	0	0	0

Table 4: Operations applied / proposed over the ten rounds, by type.

Limitations

Scale and variance. Each test set has at most 100 sentences in four to seven documents, and every configuration was run once, in one order of training groups, with cached answers. The bootstrap intervals describe variation over test documents, not over runs of the builder or of the annotators; repeated trajectories and other group orders are needed to estimate that variance.

Models. The builder (Sonnet) and the main annotator (Haiku) are from one model family. Within our API budget for GPT-6.1-sol (through OpenRouter, without prompt caching) we ran all seed configurations with GPT, the GPT guideline baseline for TimeML only, and the transfer of H_{10} to GPT/GPT for Chinese and for two English documents; the other GPT cells are marked n/a. Claude models ran through the Claude Agent SDK on a subscription and GPT through OpenRouter; providers' defaults and versions may change.

Data. The English training groups come from only three fully annotated documents; most of the released English UMR data are partial conversions without aspect or document-level relations. The Chinese gold we score against differs from the shared task's own gold (Appendix A), so we report both. The TimeML results depend strongly on which gold the builder sees (§5.3). Test documents may occur in the models' pretraining data.

Admission. Operations are admitted by machine checks only; nothing tests an edit before it is kept, so a round can make the harness worse on unseen data (Figure 2). A held-in check of each edit, and people adjudicating the gold inconsistencies the builder reports, are natural additions.

Human influence. The seed builder's prompt, the reflection prompt, the operation set, the judges and the admission rules were designed by the authors. During the runs we changed the admission code twice (rule steps get one repair attempt; helper functions are allowed in tools) and restarted the trajectories from scratch each time, and we replaced the English and TimeML training pools once after finding training documents without document-level annotation.

Ethics Statement

Automating annotation can reduce the paid work available to annotators. We see AnnoForge as a way to start an annotation project from a working draft annotator, with people deciding conventions and adjudicating the inconsistencies the builder surfaces. UMR data are used under their CC licenses; TimeBank and AQUAINT text (LDC) is not redistributed. Model calls went to commercial APIs. Generative AI was used to run the experiments and to draft and edit this paper.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations (ICLR)*.

Steven Bethard. 2013. ClearTK-TimeML: A minimalist approach to TempEval 2013. In *Second Joint Conference on Lexical and Computational Semantics (*SEM), Volume 2: Proceedings of the Seventh International Workshop on Semantic Evaluation (SemEval 2013)*, pages 10–14, Atlanta, Georgia, USA. Association for Computational Linguistics.

Adrien Bibal, Nathaniel Gerlek, Goran Muric, Elizabeth Boschee, Steven C. Fincke, Mike Ross, and Steven N. Minton. 2025. Automating annotation guideline improvements using LLMs: A case study. In *Proceedings of Context and Meaning: Navigating Disagreements in NLP Annotation*, pages 129–144, Abu Dhabi, UAE. International Committee on Computational Linguistics.

Julia Bonn, Claire Bonial, Matt Buchholz, and 1 others. 2026. Uniform meaning representation 2.2. LINDAT/CLARIAH-CZ digital library at the Institute of Formal and Applied Linguistics (ÚFAL), Faculty of Mathematics and Physics, Charles University

(publisher: UMR Consortium). Includes the DMR 2026 shared-task training data, gold test data and participating systems' outputs.

Julia Bonn, Matthew Buchholz, Jayeol Chun, Andrew Cowell, William Croft, Lukas Denk, Sijia Ge, Jan Hajič, Kenneth Lai, James H. Martin, Skatje Myers, Alexis Palmer, Martha Palmer, Claire Benét Post, James Pustejovsky, Kristine Stenzel, Haibo Sun, Zdeňka Urešová, Rosa Vallejos, and 4 others. 2024. Building a broad infrastructure for uniform meaning representations. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 2537–2547, Torino, Italia. ELRA and ICCL.

Eric Brill. 1995. Transformation-based error-driven learning and natural language processing: A case study in part-of-speech tagging. *Computational Linguistics*, 21(4):543–565.

Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2024. Large language models as tool makers. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Ching-An Cheng, Allen Nie, and Adith Swaminathan. 2024. Trace is the next AutoDiff: Generative optimization with rich feedback, execution traces, and LLMs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*.

Jayeol Chun and Nianwen Xue. 2024. Uniform meaning representation parsing as a pipelined approach. In *Proceedings of TextGraphs-17: Graph-based Methods for Natural Language Processing*, pages 40–52.

Rémi de Vergnette and Maxime Amblard. 2026. Sema system for the DMR 2026 shared task: Multistage UMR parsing with Qwen3-4B. In *Proceedings of The Seventh International Workshop on Designing Meaning Representations (DMR 2026) @ LREC 2026*, pages 155–159, Palma, Mallorca (Spain). ELRA Language Resources Association (ELRA).

Allyson Ettinger, Jena Hwang, Valentina Pyatkin, Chandra Bhagavatula, and Yejin Choi. 2023. "you are an expert linguistic annotator": Limits of LLMs as analyzers of abstract meaning representation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 8250–8263.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2024. Promptbreeder: Self-referential self-improvement via prompt evolution. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Federica Gamba, Alexis Palmer, and Daniel Zeman. 2025. Bootstrapping UMRs from Universal Dependencies for scalable multilingual annotation. In *Proceedings of the 19th Linguistic Annotation Workshop (LAW-XIX-2025)*, pages 126–136, Vienna, Austria. Association for Computational Linguistics.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120.

Feng Gu, Zongxia Li, Carlos R. Colon, Benjamin Evans, Ishani Mondal, and Jordan Lee Boyd-Graber. 2026. Large language models are effective human annotation assistants, but not good independent annotators. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 71–89, San Diego, California, United States. Association for Computational Linguistics.

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2024. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Andrew Halterman and Katherine A. Keith. 2026a. Codebook LLMs: Evaluating LLMs as measurement tools for political science concepts. *Political Analysis*, 34(2):188–204.

Andrew Halterman and Katherine A. Keith. 2026b. What is a protest anyway? codebook conceptualization is still a first-order concern in LLM-era classification. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2043–2059, San Diego, California, United States. Association for Computational Linguistics.

Xingwei He, Zhenghao Lin, Yeyun Gong, A-Long Jin, Hang Zhang, Chen Lin, Jian Jiao, Siu Ming Yiu, Nan Duan, and Weizhu Chen. 2024. AnnoLLM: Making large language models to be better crowdsourced annotators. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 165–190.

Zeyu He, Zhuqian Zhou, Kirk Vanacore, Rene F. Kizilcec, and Ting-Hao 'Kenneth' Huang. 2026. Experts rise where LLMs disagree: Using cross-model disagreement to target expert effort in LLM codebook revision for large-scale annotation. arXiv:2609.26926. *Preprint*, arXiv:2609.26926.

Johannes Heinecke and Munshi Asadullah. 2026. Orange @ UMR parsing shared task. In *Proceedings of The Seventh International Workshop on Designing Meaning Representations (DMR 2026) @ LREC 2026*, pages 148–154, Palma, Mallorca (Spain). ELRA Language Resources Association (ELRA).

Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Shizhou Huang, Bo Xu, Yang Yu, Changqun Li, and Xin Alex Lin. 2025. GuideNER: Annotation guidelines are better than examples for in-context named entity recognition. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 24159–24166.

Daniel Jaroslawicz, Brendan Whiting, Parth Shah, and Karime Maamari. 2025. How many instructions can LLMs follow at once? arXiv:2507.11538. *Preprint*, arXiv:2507.11538.

Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. 2024. When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics*, 12:1417–1440.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Kon Woo Kim, Rezarta Islamaj, Jin-Dong Kim, Florian Boudin, and Akiko Aizawa. 2025. Repurposing annotation guidelines to instruct LLM annotators: A case study. In *International Conference on Applications of Natural Language to Information Systems (NLDB 2025), Springer LNCS*, pages 140–151.

Kon Woo Kim, Jin-Dong Kim, and Akiko Aizawa. 2026. Refining and reusing annotation guidelines for LLM annotation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 37951–37964, San Diego, California, United States. Association for Computational Linguistics.

Siun Kim and Hyung-Jin Yoon. 2026. DiZiNER: Disagreement-guided instruction refinement via simulating pilot annotation for zero-shot named entity recognition. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17498–17519, San Diego, California, United States. Association for Computational Linguistics.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-harness: End-to-end optimization of model harnesses. arXiv:2603.28052. *Preprint*, arXiv:2603.28052.

Mosh Levy, Alon Jacoby, and Yoav Goldberg. 2024. Same task, more tokens: the impact of input length on the reasoning performance of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15339–15353, Bangkok, Thailand. Association for Computational Linguistics.

Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. 2026a. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. arXiv:2604.25850. *Preprint*, arXiv:2604.25850.

Minhua Lin, Juncheng Wu, Zijun Wang, Zhan Shi, Yisi Sang, Bing He, Zewen Liu, Tianxin Wei, Zongyu Wu, Zhiwei Zhang, Dakuo Wang, Xiang Zhang, Benoit Dumoulin, Cihang Xie, Yuyin Zhou, Suhang Wang, and Hanqing Lu. 2026b. Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving LLM agents. arXiv:2605.30621. *Preprint*, arXiv:2605.30621.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Emma Markle, Javier Gutierrez Bach, and Shira Wein. 2026. SETUP: Sentence-level English-to-uniform meaning representation parser. In *Proceedings of the Fifteenth Language Resources and Evaluation Conference*, pages 9710–9721, Palma de Mallorca, Spain. ELRA Language Resource Association.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131. *Preprint*, arXiv:2506.13131.

Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366.

Martha Palmer, Daniel Gildea, and Paul Kingsbury. 2005. The proposition bank: An annotated corpus of

semantic roles. *Computational Linguistics*, 31(1):71–106.

Liudas Panavas, Sebastian Minus, Bradley Monton, Derek Ray, Suhaas Garre, Sushant Mehta, and Edwin Chen. 2026. HANDBOOK.md: A benchmark for long-context agentic instruction following. arXiv:2607.25398. *Preprint*, arXiv:2607.25398.

Chaoxu Pang, Yixuan Cao, Qiang Ding, and Ping Luo. 2023. Guideline learning for in-context information extraction. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15372–15389, Singapore. Association for Computational Linguistics.

Reid Pryzant, Dan Iter, Jerry Li, Yin Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7957–7968.

James Pustejovsky, José Castaño, Robert Ingria, Roser Saurí, Robert Gaizauskas, Andrea Setzer, Graham Katz, and Dragomir Radev. 2003. TimeML: Robust specification of event and temporal expressions in text. In *AAAI Spring Symposium 2003 (AAAI Technical Report SS-03-07)*.

James Pustejovsky and Amber Stubbs. 2012. *Natural Language Annotation for Machine Learning: A Guide to Corpus-Building for Applications*. O'Reilly Media.

James Pustejovsky, Marc Verhagen, Roser Saurí, Jessica Littman, Robert Gaizauskas, Graham Katz, Inderjeet Mani, Robert Knippen, and Andrea Setzer. 2006. TimeBank 1.2. Linguistic Data Consortium, Philadelphia. LDC Catalog No. LDC2006T08.

Cheng Qian, Chi Han, Yi Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. 2023. CREATOR: Tool creation for disentangling abstract and concrete reasoning of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 6922–6939.

Oscar Sainz, Iker García-Ferrero, Rodrigo Agerri, Oier Lopez de Lacalle, German Rigau, and Eneko Agirre. 2024. GoLLIE: Annotation guidelines improve zero-shot information-extraction. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Roser Saurí, Jessica Littman, Bob Knippen, Robert Gaizauskas, Andrea Setzer, and James Pustejovsky. 2006. TimeML annotation guidelines, version 1.2.1. https://timeml.github.io/site/publications/timeMLdocs/annguide_1.2.1.pdf. Dated January 31, 2006.

Jürgen Schmidhuber. 2003. Goedel machines: Self-referential universal problem solvers making provably optimal self-improvements. arXiv:cs/0309048. *Preprint*, arXiv:cs/0309048.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Prateek Singh. 2026. MAGE: Understanding stability-performance trade-offs in multi-component prompt optimization. arXiv:2607.11944. *Preprint*, arXiv:2607.11944.

Saurabh Srivastava, Sweta Pati, and Ziyu Yao. 2025. Instruction-tuning LLMs for event extraction with annotation guidelines. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 13055–13071.

Jan Štěpánek, Daniel Zeman, Markéta Lopatková, Federica Gamba, Hana Hledíková, and Nianwen Xue. 2026. First shared task on UMR parsing. In *Proceedings of The Seventh International Workshop on Designing Meaning Representations (DMR 2026) @ LREC 2026*, pages 136–147, Palma, Mallorca (Spain). ELRA Language Resources Association (ELRA).

Haibo Sun, Jayeol Chun, and Nianwen Xue. 2025. AnCast++: Document-level evaluation of graph-based meaning representations. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19642–19654.

Haibo Sun and Nianwen Xue. 2024. Anchor and broadcast: An efficient concept alignment approach for evaluation of semantic graphs. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 1052–1062, Torino, Italia. ELRA and ICCL.

Haibo Sun, Nianwen Xue, Jin Zhao, Liulu Yue, Yao Sun, Keer Xu, and Jiawei Wu. 2024. Chinese UMR annotation: Can LLMs help? In *Proceedings of the Fifth International Workshop on Designing Meaning Representations @ LREC-COLING 2024*, pages 131–139, Torino, Italia. ELRA and ICCL.

Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2026. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7080–7106.

Zhen Tan, Dawei Li, Song Wang, Alimohammad Beigi, Bohan Jiang, Amrita Bhattacharjee, Mansooreh Karami, Jundong Li, Lu Cheng, and Huan Liu. 2024. Large language models for data annotation and synthesis: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 930–957.

Naushad UzZaman and James Allen. 2011. Temporal evaluation. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics:*

Human Language Technologies, pages 351–356, Portland, Oregon, USA. Association for Computational Linguistics.

Naushad UzZaman, Hector Llorens, Leon Derczynski, James Allen, Marc Verhagen, and James Pustejovsky. 2013. SemEval-2013 task 1: TempEval-3: Evaluating time expressions, events, and temporal relations. In *Second Joint Conference on Lexical and Computational Semantics (*SEM), Volume 2: Proceedings of the Seventh International Workshop on Semantic Evaluation (SemEval 2013)*, pages 1–9, Atlanta, Georgia, USA. Association for Computational Linguistics.

Jens E. L. Van Gysel, Meagan Vigus, Jayeol Chun, Kenneth Lai, Sarah Moeller, Jiarui Yao, Tim O'Gorman, Andrew Cowell, William Croft, Chu-Ren Huang, Jan Hajič, James H. Martin, Stephan Oepen, Martha Palmer, James Pustejovsky, Rosa Vallejos, and Nianwen Xue. 2021. Designing a uniform meaning representation for natural language processing. *KI - Künstliche Intelligenz*, 35(3–4):343–360.

Marc Verhagen, Roser Saurí, Tommaso Caselli, and James Pustejovsky. 2010. SemEval-2010 task 13: TempEval-2. In *Proceedings of the 5th International Workshop on Semantic Evaluation*, pages 57–62, Uppsala, Sweden. Association for Computational Linguistics.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024a. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*.

Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. 2021. Want to reduce labeling cost? GPT-3 can help. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4195–4205, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Wenxiao Wang, Priyatham Kattakinda, and Soheil Feizi. 2026. Do agent optimizers compound? a continual-learning evaluation on terminal-bench 2.0. arXiv:2607.14004. *Preprint*, arXiv:2607.14004.

Zhiruo Wang, Graham Neubig, and Daniel Fried. 2024b. TroVE: Inducing verifiable and efficient toolboxes for solving programmatic tasks. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 51177–51191. PMLR.

Kangda Wei, Aayush Gautam, and Ruihong Huang. 2024. Are LLMs good annotators for discourse-level event relation extraction? In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1–19, Miami, Florida, USA. Association for Computational Linguistics.

Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhuang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. 2026. RRSI: Regularized recursive self-improvement of agent harnesses. arXiv:2609.24972. *Preprint*, arXiv:2609.24972.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. 2025. Gödel agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27890–27913, Vienna, Austria. Association for Computational Linguistics.

Boqin Yuan, Xiaoyi Gu, Fiona Li, Chang Wan, Angel Hsing-Chi Hwang, and Jieyu Zhao. 2026. CALICO: A human-centered, codebook-aligned system for annotation. arXiv:2609.14726. *Preprint*, arXiv:2609.14726.

Lifan Yuan, Yangyi Chen, Xingyao Wang, Yi R. Fung, Hao Peng, and Heng Ji. 2024. CRAFT: Customizing LLMs by creating and retrieving from specialized toolsets. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by backpropagating language model feedback. *Nature*, 639(8055):609–616.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2024. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling (COLM)*.

Daniel Zeman and Federica Gamba. 2026. Word alignment-based evaluation of uniform meaning representations. arXiv:2603.26401. *Preprint*, arXiv:2603.26401.

Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. 2026a. Self-harness: Harnesses that improve themselves. arXiv:2606.09498. *Preprint*, arXiv:2606.09498.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2026b. Darwin Gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations (ICLR)*.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026c. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations (ICLR)*.

Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. 2025. SkillWeaver: Web agents can self-improve by discovering and honing skills. arXiv:2504.07079. *Preprint*, arXiv:2504.07079.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large language models are human-level prompt engineers. In *The Eleventh International Conference on Learning Representations (ICLR)*.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. 2024. GPTSwarm: Language agents as optimizable graphs. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 62743–62767. PMLR.

Caleb Ziems, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang. 2024. Can large language models transform computational social science? *Computational Linguistics*, 50(1):237–291.

A Data

UMR test documents. The shared task's test set (Štěpánek et al., 2026) has 15 Chinese and 5 English documents. We use those whose gold we have in a cleaned version: Chinese zh-0021–0024 and zh-0027–0035 (not zh-0026, zh-0036) and English en-0005–0008 (not en-0000). Shuffled with seed 20261012 and taken whole while the total stays at most 100 sentences, the Chinese test documents are 0023, 0027, 0028, 0030, 0031, 0032 and 0035 (99 sentences) and the English ones are en-0005, 0006, 0007 and 0008 (95 sentences). Tokens are identical to the shared task's blind test files. Our cleaned gold and the shared task's own test gold (UMR 2.2) agree at ju:mætʃ 99.0 on the English documents but only 71.7 on the Chinese ones, which differ in alignment and in a number of annotation decisions; the main metric (AnCast++ F) uses the cleaned gold, ju:mætʃ the official gold. The two annotations of the same English text in the test set (en-0005 and en-0007) agree at ju:mætʃ 81.8 with each other, a rough ceiling for any system on that text.

UMR training groups. Chinese: ten documents of 0001–0018, one random window of five sentences each. English: the release is mostly partial AMR-to-UMR conversions (few aspect attributes, no document-level relations) or Minecraft dialogues; only three documents not sharing text with the test set have full sentence and document-level annotation (english_umr-0578 = wsj_0003, 0579 = wsj_0004, 0580 = the Pear Story), and they give the ten non-overlapping windows. The shared task's clean English training files 0001 and 0003 are the texts of test documents en-0006 and en-0008, annotated again, so we exclude them.

TimeML. Test: CNN_20130322_1003, CNN_20130322_248, CNN_20130322_314, WSJ_20130321_1145, WSJ_20130322_804, bbc_20130322_1353 and nyt_20130321_cyprus (96 sentences, scored against the official keys). Sentences are split at sentence-final punctuation followed by a capital, a digit or a quote, and at blank lines, never inside a gold tag. Training groups come either from TBAQ-cleaned (the TempEval-3 training data) or from the 13 platinum documents that are not test documents. A window is scored on its own text: its tags, and TLINKs between them or to the document creation time.

B The Judges

umrjudge. AnCast++ aligns each predicted node to at most one gold node. The judge then scores 25 checks in 11 groups over the aligned graphs; each check yields units with a verdict (correct, partial, wrong, missing, extra) and a credit, and reports F over its units.

Group	Checks
a nodes	`event_found`, `event_concept`, `entity_found`, `entity_concept`, `granularity`
b senses	`event_sense`
c attributes	`aspect`, `modstr`, `polarity`, `refer`, `other_attributes`
d arguments	`core_args_event`, `core_args_abstract`, `peripheral_event`, `peripheral_entity`
e role labels	`core_labels`, `peripheral_labels`
f names	`name_tokens`, `wiki`
g reentrancy	`reentrancy`
h–k document	`temporal`, `modal`, `coref_chain`, `coref_subset`, `doc_format`

Table 5: The checks of umrjudge.

timemljudge. Predicted and gold `TIMEX3` and `EVENT` tags are matched one-to-one by character overlap, largest first. Times: `timex_found` (relaxed extent), `timex_extent` (exact extent of a matched pair), `timex_type`, `timex_value`.

Events: `event_found`, `event_extent`, `event_class`, and the instance attributes `event_tense`, `event_aspect`, `event_polarity`, `event_modality`. Temporal links, with predicted ends mapped to their matched gold entities and B–A links turned around: for event–time in the same sentence and in different sentences, event–document time, event–event in the same and in different sentences, and time–time, a `pair` check (are the right pairs linked) and a `label` check (the relation of a pair linked on both sides). The first group follows TempEval-3's entity measures (UzZaman et al., 2013), the link categories TempEval-2's tasks C–F (Verhagen et al., 2010); unlike the official temporal-awareness score (UzZaman and Allen, 2011), the link checks use no temporal closure.

C Prompts

Output format (UMR). Every annotating step receives this system message (the TimeML one is analogous): *"You annotate {Chinese|English} text in Uniform Meaning Representation (UMR), one sentence at a time. Output format. The annotation of sentence number N is written exactly as a .umr file writes one sentence, in three sections:* `# sentence level graph:` *(a PENMAN graph; every variable is sN followed by a letter and optional digits)* `# alignment:` *(one line per variable:* `variable: start-end`*, 1-based word positions;* `0-0` *when aligned to no word)* `# document level annotation: (sNs0 / sentence :temporal (. . .) :modal (. . .) :coref (. . .))`. *In the document level annotation, a and b are variables of this sentence or of earlier sentences, or constants; leave out a group that has no relation."*

Seed. The builder receives the description of the three steps, the output format and the whole guideline, and is asked for each step's instructions, "at most 8,000 characters, in English, organized as a compact checklist a model can follow: what to annotate and in which order, the decisions that matter most for this scheme and how the guidelines settle them, the closed lists of labels and values that apply, and the frequent mistakes to avoid".

Reflection. The reflection prompt (`af/rsi.py` in the code) describes the annotator, the resource files and the operations with their constraints, then gives the current harness, the history of earlier rounds (training scores, applied and rejected operations), and the group's sentences with gold, every step's output and the judge's incorrect units. It ends: "problems: the error patterns of this group, most costly first, each with the step it should be fixed in. reflection: why they happen, and what you will change and why. operations: the changes, in the order to apply them. suggestions: anything else." Suggestions are never implemented, and the prompt says so.

D Reproduction

All code, harness versions, per-round reflections, operations, diffs and scores are in the experiment folder `experiment/annoforge` of the code repository [anonymize for review]; `af/run.py` annotates and scores one harness version, `af/rsi.py` runs a trajectory, `af/sota.py` re-scores earlier systems, and `analysis/annoforge.py` in the paper repository makes every table and figure. Claude models ran through the Claude Agent SDK, GPT-6.1-sol through OpenRouter; every answer is cached, so a run can be replayed exactly. TimeBank and AQUAINT text is not redistributed.

E Round-by-Round Records

Tables 6–8 list every round of the three main trajectories: the main metric of the training group before and after the round (F for UMR, TLINK F1 for TimeML), the test score of the new version, the operations applied out of those proposed, the total size of the skills, and the number of model steps, rule steps and tools.

	Group n		Test	Ops	Skills	Model steps /
	H_{n-1}	H_n	H_n	applied	(K chars)	rules / tools
H_0	–	–	46.4	–	17.4	3 / 0 / 0
H_1	56.7	73.7	47.2	4/4	13.0	3 / 1 / 0
H_2	42.1	71.5	51.3	4/4	12.0	3 / 1 / 1
H_3	64.4	82.2	52.6	10/10	15.6	3 / 2 / 1
H_4	58.6	76.0	52.2	5/5	17.8	3 / 3 / 1
H_5	58.2	70.2	53.5	5/6	18.9	3 / 4 / 1
H_6	59.6	69.5	52.3	18/18	19.6	3 / 5 / 1
H_7	66.5	71.1	52.5	9/11	20.1	3 / 6 / 1
H_8	68.7	83.8	53.2	9/9	21.7	3 / 7 / 1
H_9	53.7	67.2	53.2	12/13	23.6	3 / 8 / 1
H_{10}	44.3	56.3	52.4	8/8	26.4	3 / 10 / 1

Table 6: Rounds of AnnoForge on Chinese UMR (AnCast++ F).

	Group n		Test	Ops	Skills	Model steps /
	H_{n-1}	H_n	H_n	applied	(K chars)	rules / tools
H_0	–	–	40.9	–	17.9	3 / 0 / 0
H_1	27.6	91.1	44.4	4/6	14.4	3 / 0 / 1
H_2	28.2	51.0	49.3	14/15	17.0	3 / 0 / 1
H_3	43.5	51.3	51.0	11/11	18.4	3 / 1 / 1
H_4	42.5	63.1	50.6	4/5	19.3	3 / 1 / 1
H_5	38.9	54.1	52.3	6/6	21.8	3 / 2 / 1
H_6	48.4	68.8	53.2	10/11	24.8	3 / 3 / 1
H_7	76.3	88.6	53.5	7/7	26.3	3 / 3 / 1
H_8	52.4	60.4	51.7	7/9	26.0	3 / 3 / 1
H_9	58.9	68.0	51.7	7/8	26.9	3 / 3 / 1
H_{10}	61.9	73.7	52.2	14/14	27.9	3 / 4 / 1

Table 7: Rounds of ANNOFORGE on English UMR (AnCast++ F).

	Group n		Test	Ops	Skills	Model steps /
	H_{n-1}	H_n	H_n	applied	(K chars)	rules / tools
H_0	–	–	39.5	–	17.2	3 / 0 / 0
H_1	57.8	61.8	38.7	13/15	19.9	3 / 0 / 0
H_2	17.8	40.0	40.3	11/11	21.3	3 / 0 / 0
H_3	36.7	57.0	38.6	17/17	22.5	3 / 0 / 0
H_4	32.4	50.0	37.7	13/15	23.6	3 / 0 / 0
H_5	47.1	47.9	35.4	10/11	24.2	3 / 0 / 0
H_6	45.7	71.8	39.1	13/13	24.7	3 / 0 / 0
H_7	22.4	37.5	37.3	14/15	25.7	3 / 0 / 0
H_8	11.1	50.0	36.4	10/11	27.3	3 / 0 / 0
H_9	57.8	71.6	39.7	11/16	28.3	3 / 0 / 0
H_{10}	39.8	40.8	39.7	9/10	28.8	3 / 0 / 0

Table 8: Rounds of ANNOFORGE on TimeML, trained on platinum documents (TLINK F1).

F Per-Check Scores

Group	Check	Units	H_0	H_{10}	Δ
a	a_event_found	471	94.4	96.6	+2.2
a	a_event_concept	421	81.2	84.9	+3.6
a	a_entity_found	1278	89.5	93.7	+4.2
a	a_entity_concept	1036	65.9	81.2	+15.3
a	a_granularity	1308	83.6	96.0	+12.3
b	b_event_sense	322	87.6	93.8	+6.2
c	c_aspect	392	74.5	74.8	+0.4
c	c_polarity	26	76.2	76.2	+0.0
c	c_refer	11	0.0	33.3	+33.3
c	c_other_attributes	280	52.6	69.9	+17.2
d	d_core_args_event	333	51.8	63.5	+11.6
d	d_core_args_abstract	50	39.2	35.2	−4.0
d	d_peripheral_event	191	35.0	40.6	+5.6
d	d_peripheral_entity	354	43.3	48.5	+5.2
e	e_core_labels	374	78.3	87.1	+8.8
e	e_peripheral_labels	309	77.3	76.7	−0.7
f	f_name_tokens	191	58.1	56.2	−1.9
f	f_wiki	97	0.0	–	–
g	g_reentrancy	340	30.0	35.5	+5.5
h	h_temporal	431	3.0	14.7	+11.6
i	i_modal	507	0.0	49.8	+49.8
j	j_coref_chain	275	13.8	17.5	+3.6
j	j_coref_subset	4	0.0	0.0	+0.0
k	k_doc_format	1437	23.0	90.5	+67.5

Table 9: umrjudge checks on the Chinese test documents, H_0 and H_{10} (Haiku / Haiku).

G Case Study Excerpts

The builder's reflection, English round 1 (verbatim, abridged).

> "Most of the lost score is format, not understanding. The skills taught a vocabulary (`:ref-person`, `Singular`, `:modstr`, `:modpred`, `:Str`, `DCT`) that the gold and the judge do not use. The critique step then reinforced it, and it even moved the temporal relation the wrong way. I will change four things. (1) I rewrite all three skills to use the gold conventions [...] (2) I add a deterministic rule step [...] that normalises attribute names and cases, converts `:modstr` into the document-level modal labels and then drops it [...] (3) I add a `roleset_lookup` tool over `propbank-en-args.txt`, so haiku picks real senses and roles instead of guessing [...] (4) I rewrite the critic to check against the same conventions."

The rule steps of (2) were rejected: after one repair attempt they still came with fewer than three tests. The builder added rule steps that passed in later rounds.

A tool and its revision (English). Round 1, the whole tool:

```
def roleset_lookup(lemma: str) -> str:
  """Look up English PropBank rolesets for a lemma
[. . . ]. Returns lines like 'dump-01 ARG0: ...
ARG1: ...' with all senses and their numbered
roles [. . . ]."""
  lem = lemma.strip().lower().replace(' ',
'-').replace('_', '-')
  text =
resources.read_resource('propbank-en-args.txt')
  out = [ln for ln in text.splitlines() if
ln.startswith(lem + '-')]
  return '\n'.join(out[:14]) if out else 'no
roleset for ' + lem
```

Round 9, reason given: "Show sense names (meaning) and roles from the XML, with fallback to base verb for derived nouns/adjectives, so the model can choose the right sense (consider-01 vs -02)." The new version reads `propbank-en/<lemma>.xml`, prints each roleset as `consider-02 [name] ARG0: . . .`, and tries *-ers*, *-ion*, *-ing* and other suffix variants when the lemma has no file.

A rule step (Chinese, round 10, the whole function).

```
def run(annotation: str, sentence: dict) -> str:
  """Ensure the document-level :modal group
contains (root :modal author), creating the group
if absent."""
  [. . . ] if PAT.search(doc): return annotation
  m = re.search(r':modal\s*\(', doc)
  if m: doc = doc[:m.end()] + '(root :modal
author)\n' + doc[m.end():]
  else: [add a :modal group with it] . . .
```

A Chinese tool answering with a convention. `lookup_frame(lemma)` returns the rolesets of a Chinese PropBank frame file, and when there is none: "NO FRAME for *lemma*: use *lemma*-00 with

Group	Check	Units	H_0	H_{10}	Δ
a	a_event_found	546	90.5	93.6	+3.1
a	a_event_concept	451	76.1	80.1	+4.1
a	a_entity_found	817	89.0	88.4	−0.6
a	a_entity_concept	655	77.4	80.2	+2.8
a	a_granularity	1046	88.4	93.9	+5.5
b	b_event_sense	315	66.7	80.1	+13.4
c	c_aspect	365	64.3	59.6	−4.7
c	c_polarity	31	45.0	61.9	+16.9
c	c_refer	114	0.0	50.0	+50.0
c	c_other_attributes	495	11.4	56.1	+44.7
d	d_core_args_event	341	41.6	54.5	+13.0
d	d_core_args_abstract	55	38.7	43.8	+5.1
d	d_peripheral_event	150	25.2	28.8	+3.7
d	d_peripheral_entity	174	42.7	43.4	+0.6
e	e_core_labels	366	76.8	84.0	+7.2
e	e_peripheral_labels	162	64.2	70.1	+5.9
f	f_name_tokens	143	71.7	79.2	+7.5
f	f_wiki	127	71.0	66.7	−4.3
g	g_reentrancy	354	18.9	33.0	+14.1
h	h_temporal	418	4.8	13.1	+8.3
i	i_modal	520	0.0	32.0	+32.0
j	j_coref_chain	427	17.2	17.4	+0.2
j	j_coref_subset	62	0.0	0.0	+0.0
k	k_doc_format	1239	24.7	86.2	+61.5

Table 10: umrjudge checks on the English test documents.

Group	Check	Units	H_0	H_{10}	Δ
a timex	timex_found	62	89.3	90.1	+0.8
a timex	timex_extent	50	92.0	90.0	−2.0
a timex	timex_type	50	94.0	98.0	+4.0
a timex	timex_value	50	88.0	88.0	+0.0
b event	event_found	384	83.8	84.3	+0.5
b event	event_extent	277	100.0	99.6	−0.4
b event	event_class	277	83.0	85.2	+2.2
b event	event_tense	277	68.2	64.8	−3.5
b event	event_aspect	277	88.8	90.2	+1.3
b event	event_polarity	277	100.0	99.6	−0.4
b event	event_modality	277	95.3	96.2	+0.9
c tlink E-T	tlink_et_same_pair	66	74.3	79.6	+5.3
c tlink E-T	tlink_et_same_label	39	89.7	87.2	−2.6
c tlink E-T	tlink_et_other_pair	15	0.0	0.0	+0.0
d tlink E-DCT	tlink_edct_pair	188	34.4	44.8	+10.4
d tlink E-DCT	tlink_edct_label	39	84.6	81.2	−3.4
e tlink E-E	tlink_ee_same_pair	230	43.0	54.5	+11.5
e tlink E-E	tlink_ee_same_label	63	74.6	65.5	−9.1
e tlink E-E	tlink_ee_other_pair	122	6.3	11.9	+5.6
e tlink E-E	tlink_ee_other_label	4	75.0	50.0	−25.0
f tlink T-T	tlink_tt_pair	15	0.0	0.0	+0.0

Table 11: timemljudge checks on the TimeML test documents (platinum-trained trajectory).

:actor (agent) / :undergoer (patient) instead of ARGn", the UMR guideline's convention for predicates without a frame file, attached by the builder to the tool's answer.